

DevSaaS Portfolio - Documentazione Completa

Architettura, sorgenti TypeScript, configurazione Firebase e guida alla riproduzione

Panoramica dello Stack Tecnologico

- **Frontend:** React 18, TypeScript, Vite
- **UI & Styling:** React-Bootstrap, Bootstrap 5, Bootstrap Icons
- **State & Auth:** React Context API, Firebase Authentication
- **Database & API:** Firebase Realtime Database, Axios (JSONPlaceholder Integration)
- **Routing:** React Router v6 (con Protected Routes)

1. Struttura del Progetto

Di seguito è riportata l'alberatura completa dei file del progetto:

```
my-saas-project/  
├── .env.local  
├── index.html  
├── package.json  
├── tsconfig.json  
├── vite.config.ts  
└── src/  
    ├── App.tsx  
    ├── main.tsx  
    ├── index.css  
    ├── vite-env.d.ts  
    ├── components/  
    │   ├── Footer.tsx  
    │   ├── NavigationBar.tsx  
    │   └── ProtectedRoute.tsx  
    ├── context/  
    │   └── AuthContext.tsx  
    ├── firebase/  
    │   └── firebaseConfig.ts  
    ├── pages/  
    │   ├── Dashboard.tsx  
    │   ├── Home.tsx  
    │   ├── Login.tsx  
    │   └── Register.tsx  
    ├── services/  
    │   └── apiService.ts
```

2. Configurazione dell'Ambiente e Build

2.1. File vite.config.ts

```
import { defineConfig } from 'vite';  
import react from '@vitejs/plugin-react';  
  
// Configurazione Vite per React e TypeScript  
export default defineConfig({  
  plugins: [react()],  
});
```

2.2. File `.env.local`

```
VITE_FIREBASE_API_KEY=AizaSyCIEAE45f7jel3404e8NrQeoveoWz0p-uA
VITE_FIREBASE_AUTH_DOMAIN=my-saas-principale.firebaseio.com
VITE_FIREBASE_DATABASE_URL=https://my-saas-principale-default-rtdb.europe-west1.firebaseio.com
VITE_FIREBASE_PROJECT_ID=my-saas-principale
VITE_FIREBASE_STORAGE_BUCKET=my-saas-principale.firebaseio.com
VITE_FIREBASE_MESSAGING_SENDER_ID=113575428119
VITE_FIREBASE_APP_ID=1:113575428119:web:930eb179528306f9c8dc52
```

2.3. File `src/index.css`

```
/* Reset CSS globale per garantire l'occupazione completa dello schermo e centraggio */
*, ::before, ::after {
  box-sizing: border-box;
}

html, body, #root {
  width: 100%;
  min-height: 100vh;
  margin: 0;
  padding: 0;
  overflow-x: hidden;
}

body {
  background-color: #f8f9fa; /* Sfondo grigio chiaro per risaltare le card bianche */
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
}
```

3. Configurazione Firebase e API Services

3.1. File `src/firebase/firebaseConfig.ts`

```
import { initializeApp } from "firebase/app";
import { getAuth, Auth } from "firebase/auth";
import { getDatabase, Database } from "firebase/database";

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY as string,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN as string,
  databaseURL: import.meta.env.VITE_FIREBASE_DATABASE_URL as string,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID as string,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET as string,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID as string,
  appId: import.meta.env.VITE_FIREBASE_APP_ID as string,
};

const app = initializeApp(firebaseConfig);

export const auth: Auth = getAuth(app);
export const db: Database = getDatabase(app);
```

3.2. Regole di Sicurezza Realtime Database

```
{
  "rules": {
    "orders": {
      ".read": "auth != null",
      "$orderId": {
        ".write": "auth != null && (!data.exists() || data.child('userId').val() === auth.uid) &&
newData.child('userId').val() === auth.uid"
      }
    }
  }
}
```

3.3. File `src/services/apiService.ts`

```
import axios from 'axios';

const API_BASE_URL = 'https://jsonplaceholder.typicode.com';

export interface DeveloperTask {
  userId: number;
  id: number;
  title: string;
  completed: boolean;
}

export const getDeveloperStats = async (): Promise<DeveloperTask[]> => {
  try {
    const response = await axios.get<DeveloperTask[]>(`${API_BASE_URL}/todos?_limit=5`);
    return response.data;
  } catch (error) {
    console.error("Errore nel recupero dei dati dall'API:", error);
    throw error;
  }
};
```

4. Gestione Stato Autenticazione (Context)

4.1. File `src/context/AuthContext.tsx`

```
import React, { createContext, useContext, useEffect, useState } from 'react';
import {
  User,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged
} from 'firebase/auth';
import { auth } from '../firebase/firebaseConfig';

interface AuthContextType {
  currentUser: User | null;
  signup: (email: string, pass: string) => Promise<any>;
  login: (email: string, pass: string) => Promise<any>;
  logout: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth deve essere usato all'interno di AuthProvider");
  return context;
};

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [currentUser, setCurrentUser] = useState<User | null>(null);
  const [loading, setLoading] = useState<boolean>(true);

  const signup = (email: string, pass: string) => createUserWithEmailAndPassword(auth, email, pass);
  const login = (email: string, pass: string) => signInWithEmailAndPassword(auth, email, pass);
  const logout = () => signOut(auth);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (user) => {
      setCurrentUser(user);
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  return (
    <AuthContext.Provider value={{ currentUser, signup, login, logout }}>
      {!loading && children}
    </AuthContext.Provider>
  );
};
```

5. Componenti di Layout e Pagine Principali

5.1. File `src/pages/Dashboard.tsx`

```

import React, { useState, useEffect } from 'react';
import { Container, Row, Col, Card, Alert, Button, ListGroup, Spinner } from 'react-bootstrap';
import { useAuth } from '../context/AuthContext';
import { useLocation, useNavigate } from 'react-router-dom';
import { db } from '../firebase/firebaseConfig';
import { ref, push, set, onValue } from 'firebase/database';
import { getDeveloperStats, DeveloperTask } from '../services/apiService';

interface Order {
  id: string;
  projectName: string;
  status: string;
  userId: string;
}

const Dashboard: React.FC = () => {
  const { currentUser, logout } = useAuth();
  const location = useLocation();
  const navigate = useNavigate();

  const [feedback, setFeedback] = useState<string>(location.state?.feedback || '');
  const [orders, setOrders] = useState<Order[]>([]);
  const [apiData, setApiData] = useState<DeveloperTask[]>([]);
  const [loadingApi, setLoadingApi] = useState<boolean>(true);

  useEffect(() => {
    const selectedProject = location.state?.selectedProject;
    if (selectedProject && currentUser) {
      const saveOrder = async () => {
        try {
          const ordersRef = ref(db, 'orders');
          const newOrderRef = push(ordersRef);
          await set(newOrderRef, {
            userId: currentUser.uid,
            userEmail: currentUser.email,
            projectName: selectedProject,
            status: 'In Elaborazione',
            createdAt: new Date().toISOString()
          });
          setFeedback(`Richiesta inviata con successo per: ${selectedProject}!`);
        } catch (error) {
          console.error("Errore nel salvataggio dell'ordine:", error);
        }
      };
      saveOrder();
    }
  }, [location.state, currentUser]);

  useEffect(() => {
    if (!currentUser) return;
    const ordersRef = ref(db, 'orders');
    const unsubscribe = onValue(ordersRef, (snapshot) => {
      const data = snapshot.val();
      if (data) {
        const userOrders: Order[] = Object.keys(data)
          .map(key => ({ id: key, ...data[key] }))
          .filter(order => order.userId === currentUser.uid);
        setOrders(userOrders);
      } else {
        setOrders([]);
      }
    });
    return () => unsubscribe();
  }, [currentUser]);

  useEffect(() => {
    getDeveloperStats()
      .then(data => { setApiData(data); setLoadingApi(false); })
      .catch(() => setLoadingApi(false));
  }, []);

```

```

const handleLogout = async () => {
  await logout();
  navigate('/login');
};

return (
  <Container className="py-5 mx-auto">
    <div className="d-flex justify-content-between align-items-center mb-4">
      <div>
        <h2 className="fw-bold mb-1">Dashboard Utente</h2>
        <p className="text-muted mb-0">Benvenuto/a, <strong>{currentUser?.email}</strong></p>
      </div>
      <Button variant="outline-danger" onClick={handleLogout}>Logout</Button>
    </div>

    {feedback && (
      <Alert variant="success" dismissible onClose={() => setFeedback('')} className="mb-4">
        {feedback}
      </Alert>
    )}

    <Row className="g-4">
      <Col lg={6} md={12}>
        <Card className="shadow-sm border-0 h-100">
          <Card.Header className="bg-primary text-white fw-bold py-3">I Tuoi Servizi Ordinati</
Card.Header>
          <Card.Body>
            {orders.length === 0 ? (
              <p className="text-muted mb-0">Non hai ancora richiesto nessun servizio.</p>
            ) : (
              <ListGroup variant="flush">
                {orders.map((order) => (
                  <ListGroup.Item key={order.id} className="d-flex justify-content-between align-items-
center py-3">
                    <span className="fw-semibold">{order.projectName}</span>
                    <span className="badge bg-warning text-dark px-3 py-2">{order.status}</span>
                  </ListGroup.Item>
                ))}
              </ListGroup>
            )}
          </Card.Body>
        </Card>
      </Col>

      <Col lg={6} md={12}>
        <Card className="shadow-sm border-0 h-100">
          <Card.Header className="bg-dark text-white fw-bold py-3">Live API Feed (Task in Lavorazione)</
Card.Header>
          <Card.Body>
            {loadingApi ? (
              <div className="text-center py-4">
                <Spinner animation="border" size="sm" className="me-2" /> Caricamento API...
              </div>
            ) : (
              <ListGroup variant="flush">
                {apiData.map((task) => (
                  <ListGroup.Item key={task.id} className="small py-2 border-0">
                    <i className={`bi ${task.completed ? 'bi-check-circle-fill text-success' : 'bi-clock
text-secondary'} me-2`} ></i>
                    {task.title}
                  </ListGroup.Item>
                ))}
              </ListGroup>
            )}
          </Card.Body>
        </Card>
      </Col>
    </Row>
  </Container>
);
};

```

```
export default Dashboard;
```

5.2. File `src/App.tsx`

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import NavigationBar from '../components/NavigationBar';
import Footer from '../components/Footer';
import ProtectedRoute from '../components/ProtectedRoute';

import Home from '../pages/Home';
import Login from '../pages/Login';
import Register from '../pages/Register';
import Dashboard from '../pages/Dashboard';

function App() {
  return (
    <AuthProvider>
      <Router>
        <div className="d-flex flex-column min-vh-100 w-100 bg-light">
          <NavigationBar />
          <main className="flex-grow-1 w-100">
            <Routes>
              <Route path="/" element={<Home />} />
              <Route path="/login" element={<Login />} />
              <Route path="/register" element={<Register />} />
              <Route path="/dashboard" element={
                <ProtectedRoute>
                  <Dashboard />
                </ProtectedRoute>
              } />
            </Routes>
          </main>
          <Footer />
        </div>
      </Router>
    </AuthProvider>
  );
}

export default App;
```

5.3. File `src/main.tsx`

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap-icons/font/bootstrap-icons.css';
import './index.css';

ReactDOM.createRoot(document.getElementById('root') as HTMLElement).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```